

Christof Teuscher

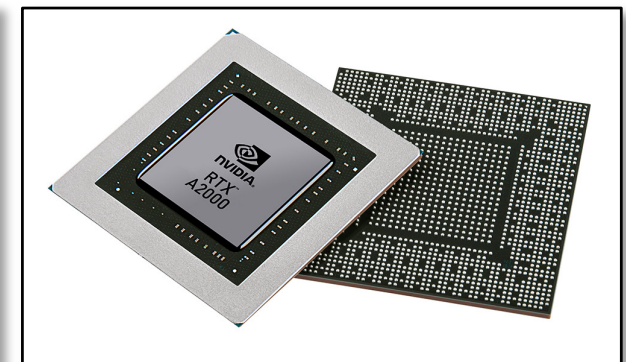
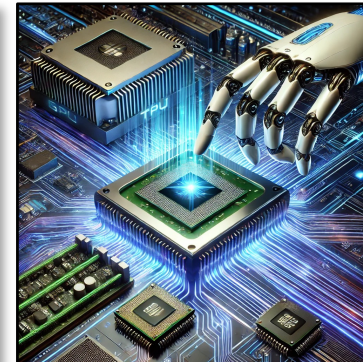
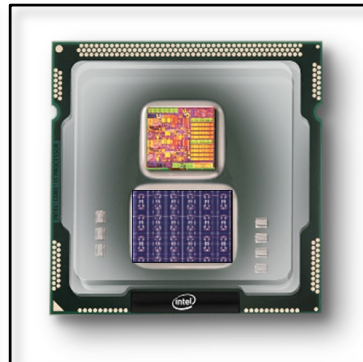
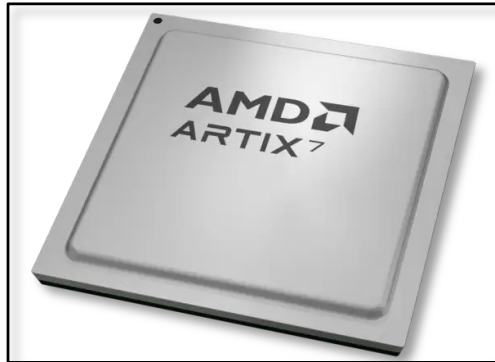
ECE 410/510: Hardware for AI and ML, Spring 2026

Codefest #3: Traffic Analysis and CUDA

Portland State University
Department of Electrical and Computer Engineering (ECE)

www.teuscher-lab.com

teuscher@pdx.edu





Today

CMAN

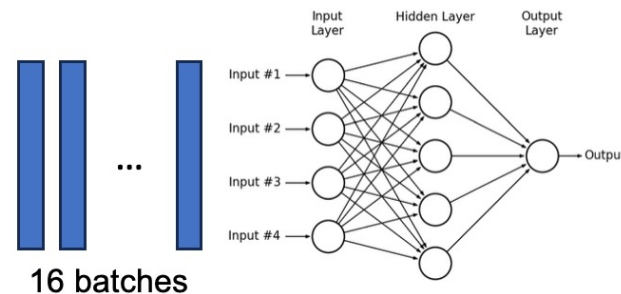
DRAM traffic analysis: naive vs. tiled matrix multiply

CLLM

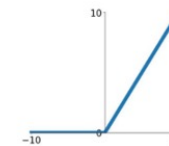
CUDA GEMM: naive vs. tiled with roofline placement

COPT

Optional challenge (+ 1 token): forward pass of a neural network on the GPU



ReLU
 $\max(0, x)$





Next week

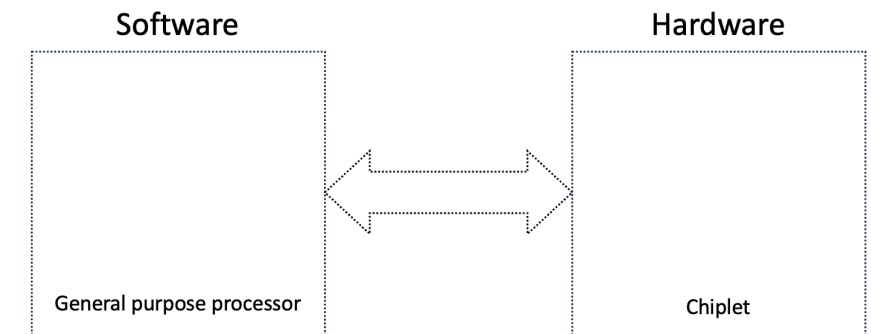
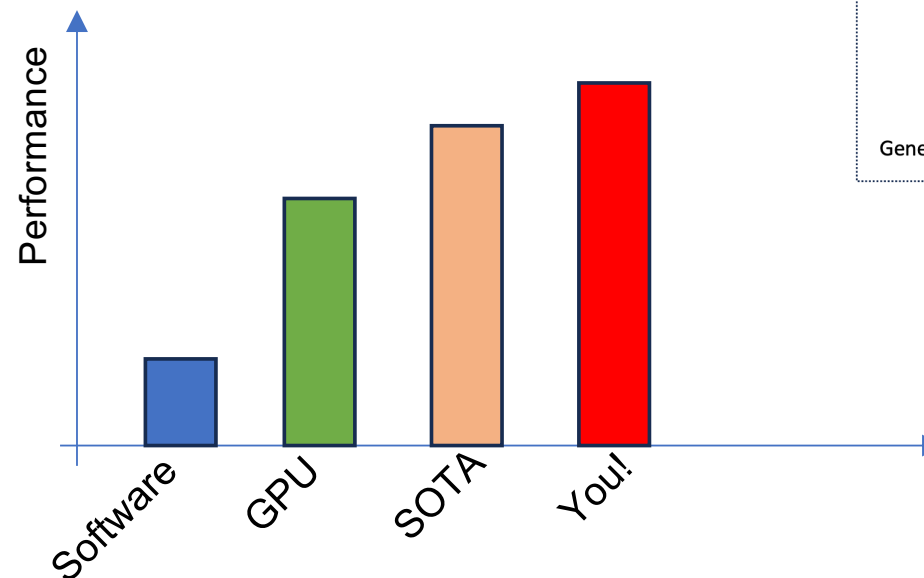
- We'll start with implementing the AI accelerator

- PyMTL (Mamba)
- PyRTL
- Cocotb
- MyHDL
- pyUVM
- yosys
- ...
- Cadence
- Synopsis

Continue to work toward M2

- Refine your ideas
- Do more profiling
- Run your code on GPU
- Re-consider the SW/HW boundary
- Compare with SOTA
- Needed precisions?

M2	Sun, May 3	First working simulation: HDL description of the compute core with a testbench demonstrating correct output for a representative input. Interface module in HDL with testbench exercising at least one write and one read/response transaction. Precision choice documented (FP32 / FP16 / BF16 / INT8 / other) with quantization error analysis if applicable.
----	------------	---



Total execution time (1)

```
import time

start_time = time.time()

# Code to be timed
def my_function():
    result = 0
    for i in range(1000000):
        result += i
    return result

my_function()

end_time = time.time()
execution_time = end_time - start_time

print(f"Execution time: {execution_time} seconds")
```

Python

```
import time

start_time = time.perf_counter()

# Code to be timed
def my_function():
    result = 0
    for i in range(1000000):
        result += i
    return result

my_function()

end_time = time.perf_counter()
execution_time = end_time - start_time

print(f"Execution time: {execution_time} seconds")
```

For more precise timing, especially for short durations, `time.perf_counter()` is recommended. It provides the highest available resolution to measure a short duration.

Total execution time (2)

Python

```
import timeit

# Code to be timed as a string
code_to_time = """
def my_function():
    result = 0
    for i in range(1000000):
        result += i
    return result
my_function()
"""

execution_time = timeit.timeit(code_to_time, number=1)

print(f"Execution time: {execution_time} seconds")
```

The timeit module is specifically designed for measuring the execution time of small code snippets.

Total execution time (3)

```
[bootis:ece_510_HW_for_AI_and_ML_2025/python/qr] cteusche% python3 qr2_profile.py qr-code-21x21.png
Execution time 1: 2.1457672119140625e-06 seconds
Execution time 2: 5.489999999885864e-07 seconds
Execution time 3: 6.220000000056736e-07 seconds
[bootis:ece_510_HW_for_AI_and_ML_2025/python/qr] cteusche% python3 qr2_profile.py qr-code-21x21.png
Execution time 1: 2.1457672119140625e-06 seconds
Execution time 2: 4.089999999790095e-07 seconds
Execution time 3: 5.87999999958141e-07 seconds
[bootis:ece_510_HW_for_AI_and_ML_2025/python/qr] cteusche% python3 qr2_profile.py qr-code-21x21.png
Execution time 1: 2.86102294921875e-06 seconds
Execution time 2: 2.548999999905866e-06 seconds
Execution time 3: 6.570000000150067e-07 seconds
[bootis:ece_510_HW_for_AI_and_ML_2025/python/qr] cteusche% python3 qr2_profile.py qr-code-21x21.png
Execution time 1: 9.5367431640625e-07 seconds
Execution time 2: 5.140000000070089e-07 seconds
Execution time 3: 5.260000000006926e-07 seconds
[bootis:ece_510_HW_for_AI_and_ML_2025/python/qr] cteusche% python3 qr2_profile.py qr-code-21x21.png
Execution time 1: 2.1457672119140625e-06 seconds
Execution time 2: 5.240000000017453e-07 seconds
Execution time 3: 5.3599999995429e-07 seconds
[bootis:ece_510_HW_for_AI_and_ML_2025/python/qr] cteusche% python3 qr2_profile.py qr-code-21x21.png
Execution time 1: 1.9073486328125e-06 seconds
Execution time 2: 5.469999999896391e-07 seconds
Execution time 3: 6.000000000172534e-07 seconds
[bootis:ece_510_HW_for_AI_and_ML_2025/python/qr] cteusche% python3 qr2_profile.py qr-code-21x21.png
Execution time 1: 9.5367431640625e-07 seconds
Execution time 2: 4.67000000003992e-07 seconds
Execution time 3: 5.95999999916032e-07 seconds
[bootis:ece_510_HW_for_AI_and_ML_2025/python/qr] cteusche% python3 qr2_profile.py qr-code-21x21.png
Execution time 1: 1.1920928955078125e-06 seconds
Execution time 2: 5.990000000177798e-07 seconds
Execution time 3: 6.559999999877775e-07 seconds
```

Total execution time (4)

```
# Execution time
start_time1 = time.time()
start_time2 = time.perf_counter()
start_time3 = timeit.default_timer()

repeats = 10

if __name__ == "__main__":
    import sys

    for r in range(repeats):

        if len(sys.argv) != 2:
            print("Usage: python qr_recognizer.py <image_path>")
            sys.exit(1)

        image_path = sys.argv[1]
        recognizer = QRCodeRecognizer()

        try:
            result = recognizer.recognize(image_path)
            print("QR Code Recognition Result:")
            print(result)
        except Exception as e:
            print(f"Error: {e}")

    end_time1 = time.time()
    end_time2 = time.perf_counter()
    execution_time1 = end_time1 - start_time1
    execution_time2 = end_time2 - start_time2
    execution_time3 = timeit.default_timer() - start_time3

    print(f"Execution time 1: {execution_time1} seconds")
    print(f"Execution time 2: {execution_time2} seconds")
    print(f"Execution time 3: {execution_time3} seconds")
```

```
Execution time 1: 1.8990371227264404 seconds
Execution time 2: 1.8990448829999997 seconds
Execution time 3: 1.8990454230000002 seconds
```


Code analysis (1)

Static Code Analysis Tools

1. Using PyCallGraph

PyCallGraph is a tool that creates call graph visualizations for Python applications:

```
python

# Install with: pip install pycallgraph graphviz
from pycallgraph import PyCallGraph
from pycallgraph.output import GraphvizOutput

def analyze_qr_code():
    with PyCallGraph(output=GraphvizOutput()):
        recognizer = QRCodeRecognizer()
        result = recognizer.recognize("path_to_qr_image.png")
        print(result)

analyze_qr_code()
```

2. Using pyan for Static Analysis

Pyan analyzes Python code and generates approximate call graphs:

```
python

# Install with: pip install pyan
# Usage from command line
# pyan qr_recognizer.py --dot > qr_graph.dot
# dot -Tpng qr_graph.dot -o qr_graph.png
```

Code analysis (2)

Runtime Analysis Tools

3. Using cProfile with Visualization

```
python

import cProfile
import pstats
from pstats import SortKey

# Profile the execution
cProfile.run('recognizer.recognize("path_to_qr_image.png")', '

# Analyze the results
p = pstats.Stats('qr_stats')
p.strip_dirs().sort_stats(SortKey.CUMULATIVE).print_stats(10)

# For visualization, you can use gprof2dot
# pip install gprof2dot
# gprof2dot -f pstats qr_stats | dot -Tpng -o qr_profile.png
```

4. Using SnakeViz for Interactive Visualization

```
python

# Install with: pip install snakeviz
# Run profiling
import cProfile
cProfile.run('recognizer.recognize("path_to_qr_image.png")', '

# Then from command line:
# snakeviz qr_stats
```

Code analysis (3)

Performance Visualization

6. Visualize Performance Bottlenecks

To visualize execution time of different components:

```
python

import time
import numpy as np
import matplotlib.pyplot as plt

def benchmark_qr_components(image_path, iterations=10):
    recognizer = QRCodeRecognizer()
    components = {}

    # Benchmark image loading
    start = time.time()
    for _ in range(iterations):
        img = recognizer.load_image(image_path)
        components["load_image"] = (time.time() - start) / iterations

    # Benchmark thresholding
    start = time.time()
```

Using Specialized Tools

For more advanced analysis, consider these specialized tools:

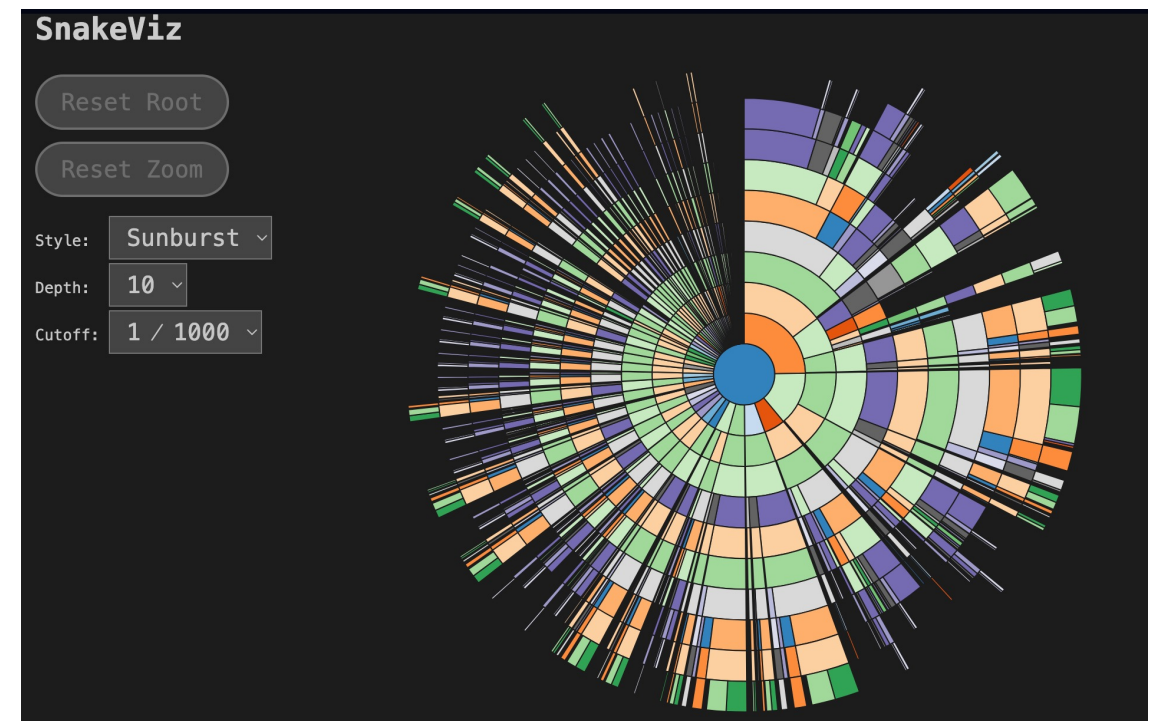
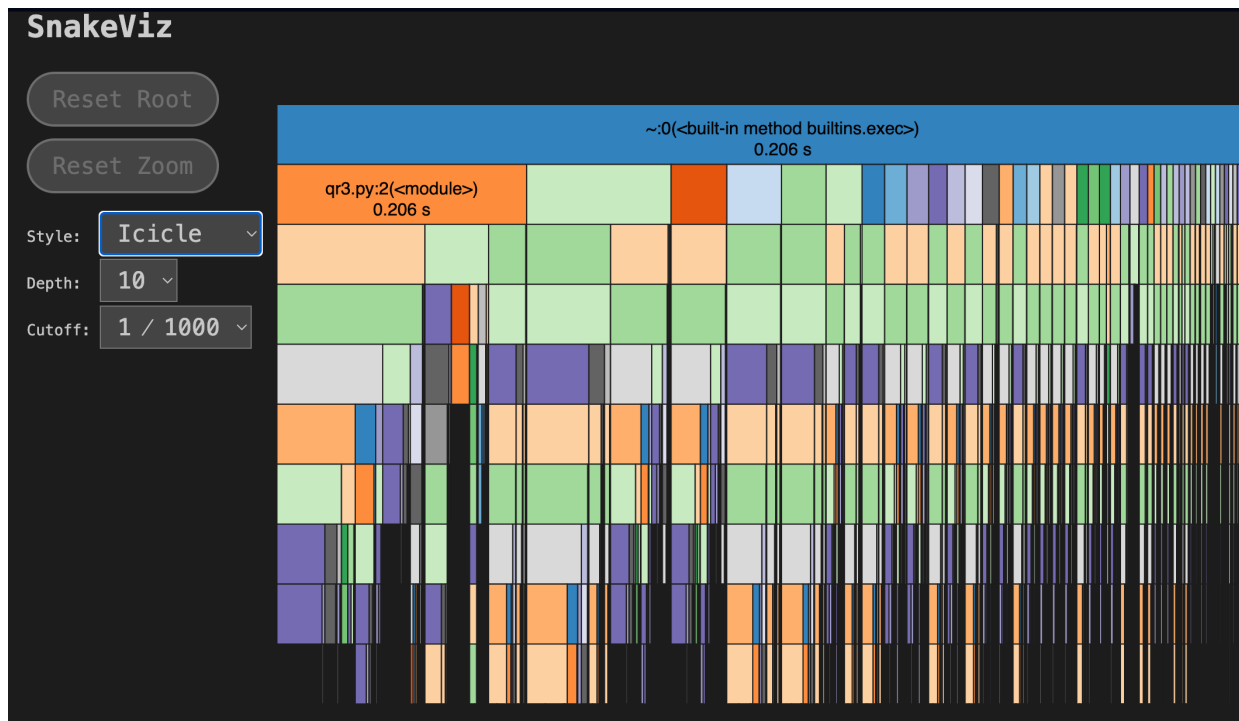
1. **Doxygen** - Can generate call graphs if properly configured
2. **CodeViz** - Works with C/C++ extensions but can be used on some Python projects
3. **Memray** - For memory profiling with visualization capabilities
4. **Py-Spy** - For sampling profiler that can generate flame graphs

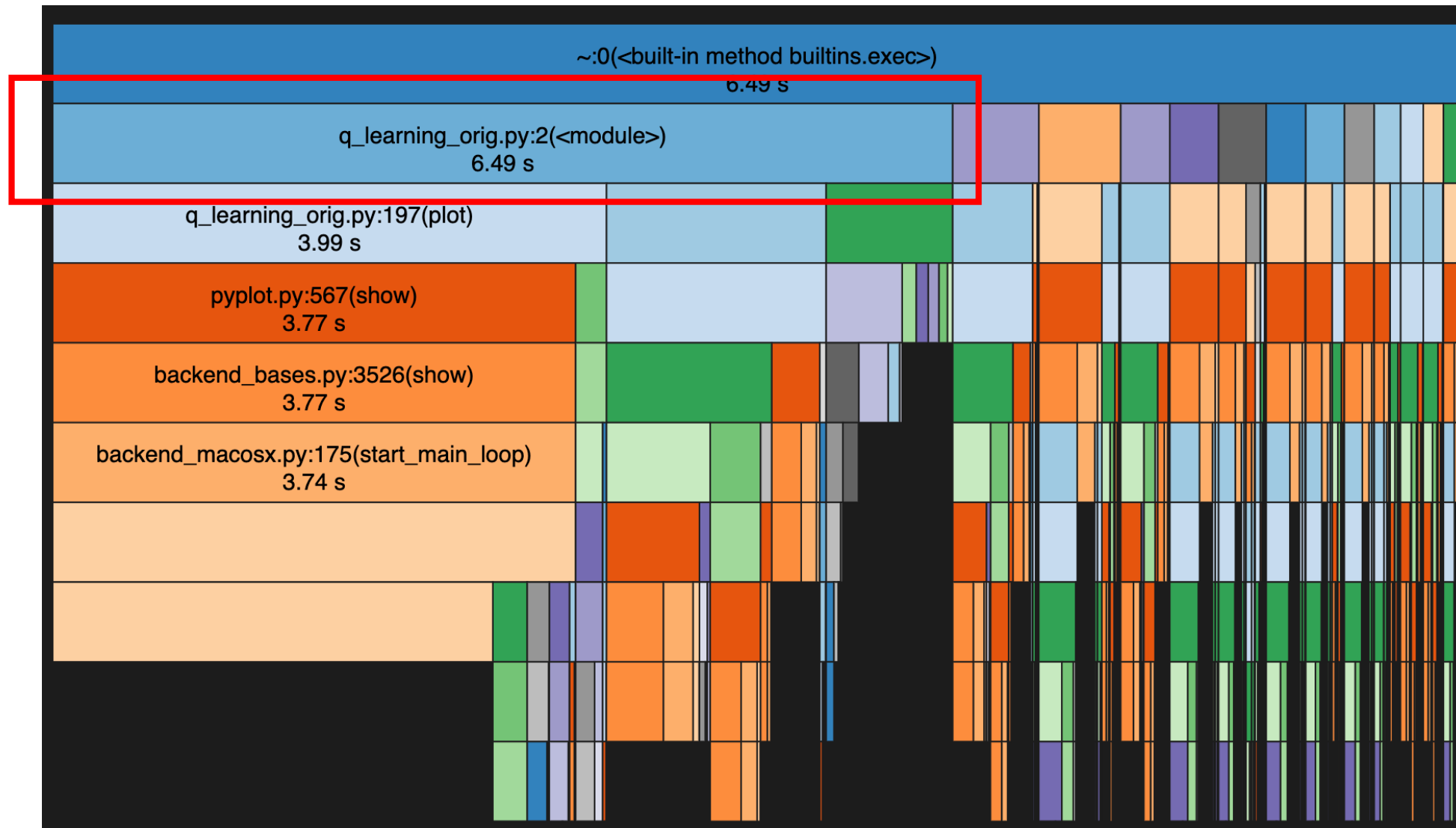
Each of these methods offers different insights into your code's structure and performance. Combining a few of them will give you the most comprehensive understanding of your QR code recognizer's data and control flow.



cProfile and SnakeViz

1. `pip install snakeviz`
2. `python3 -m cProfile -o output.prof qr_recognizer.py qr.png`
3. `snakeviz output.prof`





ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
37/36	0.03897	0.001082	0.04111	0.001142	~:0(<built-in method _imp.create_dynamic>)
153	0.01551	0.0001014	0.01551	0.0001014	~:0(<built-in method marshal.loads>)
1849	0.01443	7.803e-06	0.01443	7.803e-06	qr3.py:105(_is_potential_finder_pattern)
165	0.008252	5.001e-05	0.008252	5.001e-05	~:0(<method 'read' of '_io.BufferedReader' objects>)
153	0.006271	4.099e-05	0.006271	4.099e-05	~:0(<built-in method io.open_code>)
809	0.005534	6.84e-06	0.005534	6.84e-06	~:0(<built-in method posix.stat>)
390/389	0.00524	1.347e-05	0.02292	5.893e-05	~:0(<built-in method builtins.__build_class__>)
20	0.004723	0.0002362	0.008198	0.0004099	enum.py:179(__new__)
1	0.003492	0.003492	0.005462	0.005462	histograms.py:678(histogram)
388	0.002841	7.323e-06	0.01371	3.534e-05	<frozen importlib._bootstrap_external>:1438(find_spec)
65	0.00221	3.4e-05	0.00221	3.4e-05	~:0(<built-in method posix.getcwd>)

- **ncalls:** Total number of calls to the function. If there are two numbers, that means the function recursed and the first is the total number of calls and the second is the number of primitive (non-recursive) calls.
- **tottime:** Total time spent in the function, not including time spent in calls to sub-functions
- **percall:** `tottime` divided by `ncalls`
- **cumtime:** Cumulative time spent in this function and all sub-functions
- **percall:** `cumtime` divided by `ncalls`
- **filename:lineno(function):** File name and line number where the function is defined, and the function's name

line_profiler

1. kernprof -l q_learning_profiling.py
2. python3 -m line_profiler -rmt "q_learning_profiling.py.lprof"



```
#Q-learning Algorithm
@profile
def Q_Learning(self, episodes):
    x = 0
    #iterate through best path for each episode
    while(x < episodes):
        #check if state is end
        if self.isEnd:
            #get current reward and add to array for plot
            reward = self.State.getReward()
            self.rewards += reward
            self.plot_reward.append(self.rewards)

            #get state, assign reward to each Q_value in state
            i,j = self.State.state
            for a in self.actions:
                self.new_Q[(i,j,a)] = round(reward,3)

            #reset state
            self.State = State()
            self.isEnd = self.State.isEnd

            #set rewards to zero and iterate to next episode
            self.rewards = 0
            x+=1
        else:
            #set to arbitrary low value to compare net state actions
            mx_nxt_value = -10
            #get current state, next state, action and current reward
            next_state, action = self.Action()
            i,j = self.State.state
            reward = self.State.getReward()
            #add reward to rewards for plot
            self.rewards +=reward
```

Total time: 1.91771 s
 File: q_learning_profiling.py
 Function: Q_Learning at line 141

Line #	Hits	Time	Per Hit	% Time	Line Contents
141					@profile
142					def Q_Learning(self, episodes):
143	1	1.0	1.0	0.0	x = 0
144					#iterate through best path for each episode
145	94259	24463.0	0.3	1.3	while(x < episodes):
146					#check if state is end
147	94258	22655.0	0.2	1.2	if self.isEnd:
148					#get current reward and add to array for plot
149	10000	13540.0	1.4	0.7	reward = self.State.getReward()
150	10000	3810.0	0.4	0.2	self.rewards += reward
151	10000	5266.0	0.5	0.3	self.plot_reward.append(self.rewards)
152					
153					#get state, assign reward to each Q_value in state
154	10000	3039.0	0.3	0.2	i,j = self.State.state
155	50000	12719.0	0.3	0.7	for a in self.actions:
156	40000	26152.0	0.7	1.4	self.new_Q[(i,j,a)] = round(reward,3)
157					
158					#reset state
159	10000	9741.0	1.0	0.5	self.State = State()
160	10000	3885.0	0.4	0.2	self.isEnd = self.State.isEnd
161					
162					#set rewards to zero and iterate to next episode
163	10000	2903.0	0.3	0.2	self.rewards = 0
164	10000	3693.0	0.4	0.2	x+=1

Line #	Hits	Time	Per Hit	% Time	Line Contents
165					<code>else:</code>
166					<code>#set to arbitrary low value to compare net state actions</code>
167	84258	19736.0	0.2	1.0	<code>mx_nxt_value = -10</code>
168					<code>#get current state, next state, action and current reward</code>
169	84258	528816.0	6.3	27.6	<code>next_state, action = self.Action()</code>
170	84258	25683.0	0.3	1.3	<code>i,j = self.State.state</code>
171	84258	108610.0	1.3	5.7	<code>reward = self.State.getReward()</code>
172					<code>#add reward to rewards for plot</code>
173	84258	33549.0	0.4	1.7	<code>self.rewards +=reward</code>
174					
175					<code>#iterate through actions to find max Q value for action based on next state action</code>
176	421290	115314.0	0.3	6.0	<code>for a in self.actions:</code>
177	337032	111121.0	0.3	5.8	<code> nxtStateAction = (next_state[0], next_state[1], a)</code>
178	337032	272470.0	0.8	14.2	<code> q_value = (1-self.alpha)*self.Q[(i,j,action)] + self.alpha*(reward + self.gamma*...</code>
179					
180					<code> #find largest Q value</code>
181	337032	87482.0	0.3	4.6	<code> if q_value >= mx_nxt_value:</code>
182	212265	47640.0	0.2	2.5	<code> mx_nxt_value = q_value</code>
183					
184					<code>#next state is now current state, check if end state</code>
185	84258	95663.0	1.1	5.0	<code>self.State = State(state=next_state)</code>
186	84258	112622.0	1.3	5.9	<code>self.State.isEndFunc()</code>
187	84258	29997.0	0.4	1.6	<code>self.isEnd = self.State.isEnd</code>
188					
189					<code>#update Q values with max Q value for next state</code>
190	84258	100733.0	1.2	5.3	<code>self.new_Q[(i,j,action)] = round(mx_nxt_value,3)</code>
191					
192					<code>#copy new Q values to Q table</code>
193	94258	96267.0	1.0	5.0	<code>self.Q = self.new_Q.copy()</code>
194					<code>#print final Q table output</code>
195	1	144.0	144.0	0.0	<code>print(self.Q)</code>

```

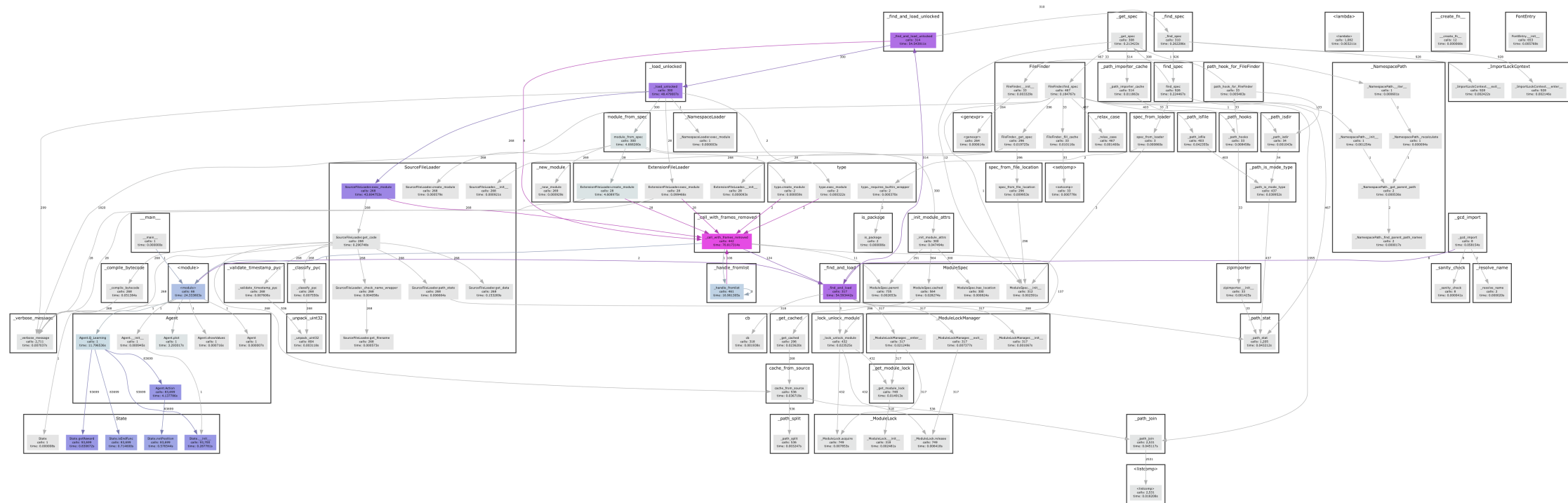
Total time: 0.828347 s
File: q_learning_profiling.py
Function: Action at line 111

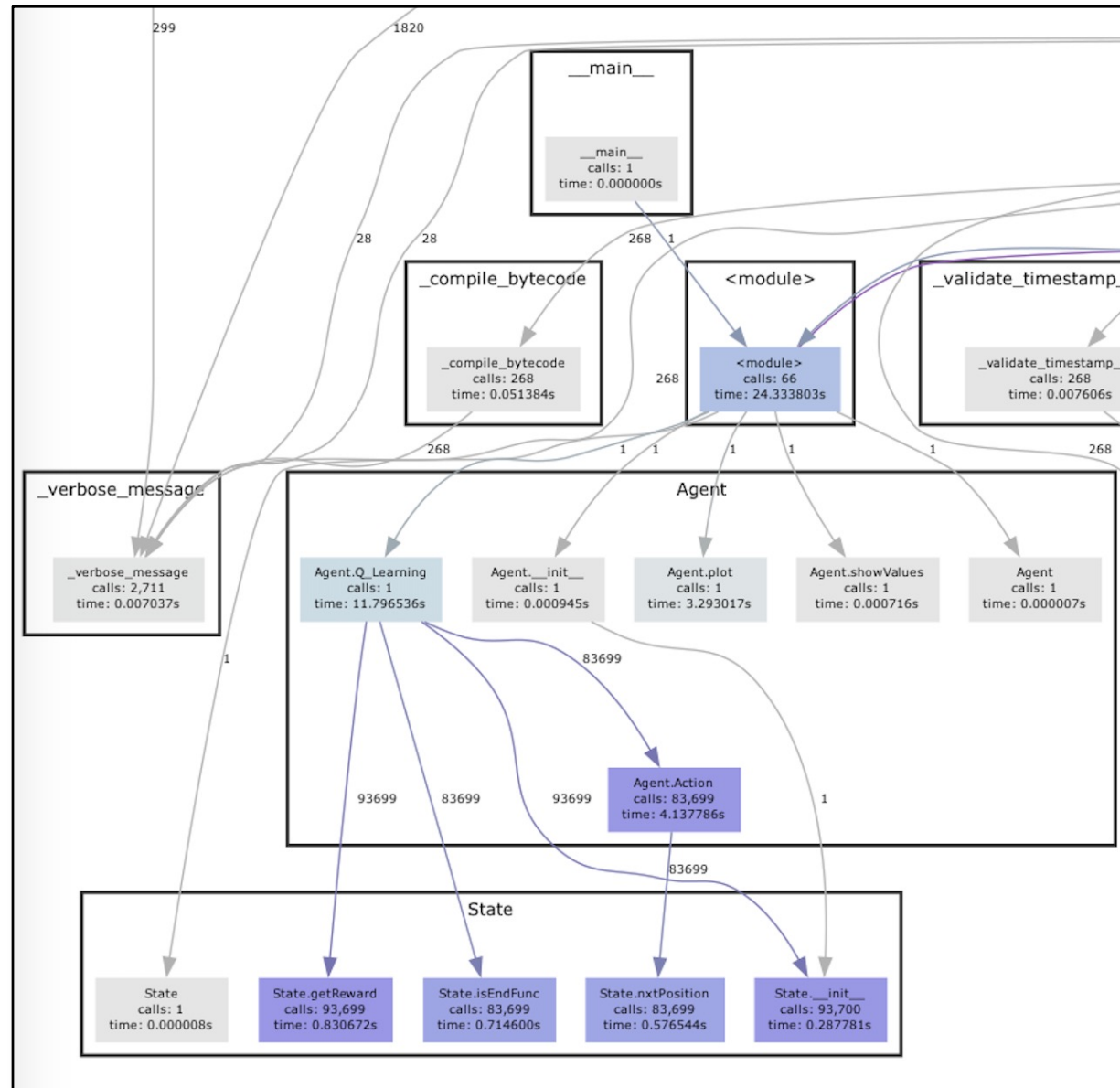
```

Line #	Hits	Time	Per Hit	% Time	Line Contents
111					@profile
112					def Action(self):
113					#random value vs epsilon
114	83741	27544.0	0.3	3.3	rnd = random.random()
115					#set arbitraty low value to compare with Q values to find max
116	83741	19066.0	0.2	2.3	mx_nxt_reward = -10
117	83741	17424.0	0.2	2.1	action = None
118					
119					#9/10 find max Q value over actions
120	83741	24591.0	0.3	3.0	if(rnd > self.epsilon):
121					#iterate through actions, find Q value and choose best
122	376785	94958.0	0.3	11.5	for k in self.actions:
123					
124	301428	80642.0	0.3	9.7	i,j = self.State.state
125					
126	301428	108115.0	0.4	13.1	nxt_reward = self.Q[(i,j, k)]
127					
128	301428	73107.0	0.2	8.8	if nxt_reward >= mx_nxt_reward:
129	180726	37599.0	0.2	4.5	action = k
130	180726	35654.0	0.2	4.3	mx_nxt_reward = nxt_reward
131					
132					#else choose random action
133					else:
134	8384	170774.0	20.4	20.6	action = np.random.choice(self.actions)
135					
136					#select the next state based on action chosen
137	83741	118470.0	1.4	14.3	position = self.State.nxtPosition(action)
138	83741	20403.0	0.2	2.5	return position,action

Pycallgraph2: Dynamic Call Graph Generation

1. Requires pycallgraph2 and graphviz
2. `pycallgraph graphviz --output-file output.png q_learning_orig.py`





ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
1	3.438	3.438	3.743	3.743	~:0(<built-in method matplotlib.backends._macosx.show>)
46/44	0.7926	0.01801	0.7994	0.01817	~:0(<built-in method _imp.create_dynamic>)
1	0.4153	0.4153	0.9049	0.9049	q_learning_orig.py:141(Q_Learning)
83522	0.1256	1.504e-06	0.2928	3.506e-06	q_learning_orig.py:111(Action)
292	0.09859	0.0003376	0.09859	0.0003376	~:0(<method 'read' of '_io.BufferedReader' objects>)
1	0.07742	0.07742	0.1346	0.1346	backend_bases.py:2673(create_with_canvas)
7178	0.06007	8.369e-06	0.107	1.491e-05	inspect.py:626(cleandoc)
8403	0.05586	6.647e-06	0.1172	1.394e-05	~:0(<method 'choice' of 'numpy.random.mtrand.RandomState' objects>)
123564	0.055	4.451e-07	0.055	4.451e-07	~:0(<built-in method builtins.round>)
292	0.04718	0.0001616	0.04718	0.0001616	~:0(<built-in method marshal.loads>)
115	0.04713	0.0004098	0.04713	0.0004098	~:0(<method 'set_message' of 'matplotlib.backends._macosx.NavigationToolba
93522	0.04602	4.921e-07	0.04602	4.921e-07	q_learning_orig.py:33(getReward)
7	0.04531	0.006473	0.04541	0.006488	~:0(<method 'draw_path' of 'matplotlib.backends._backend_agg.RendererAgg'
83522	0.04079	4.884e-07	0.04079	4.884e-07	q_learning_orig.py:53(nxtPosition)

Presentations (1)

- **Goals**
 - Share your idea
 - Share your progress
 - Hone your presentation skills
- **Schedule**
 - Presentation duration: 1 minute (strict)
 - Schedule linked on Canvas
- **Instructions**
 - Create a single (= 1 = ONE) slide on which you answer the following questions:
 - What are you trying to do? I.e., what AI/ML algorithm/workload did you pick?)
 - How have others implemented and/or accelerated this algorithm?
 - What are you doing differently/better/etc.?
 - What have you accomplished so far?
 - What will you do next and what remains to be done until you can declare success?
- Save the slide in the shared Google folder. URL on Canvas
- A strong presentation earns 1 token.

Presentations (2)

Date	Week	Time	Name
Wed, April 22	4	4:55: PM	Eraganeni, Vamsidhar Reddy (Pronoun: He/Him)
Wed, April 22	4	4:57: PM	Kamarajugadda, Venkata Sriram (Pref: Venkata Sriram)
Wed, April 22	4	4:59: PM	Boss, David (Pronoun: He/Him)
Wed, April 22	4	5:01: PM	Hynes, Kay (Pref: Kay)
Wed, April 22	4	5:03: PM	Gilbert-Croysdale, Rebecca R. (Pronoun: She/Her)
Wed, April 22	4	5:05: PM	Charian, Sai Ganesh Reddy
Wed, April 29	5	4:55: PM	Vemula, Rajeshwar Reddy (Pronoun: he/him)
Wed, April 29	5	4:57: PM	Henrichs-Tarasenkov, Fyodor P. (Pref: Fedya, Pronoun: He/him)
Wed, April 29	5	4:59: PM	Herrera Moreno, Ivan (Pref: Ivan)
Wed, April 29	5	5:01: PM	Nguyen, Andrew (Pref: Bao)
Wed, April 29	5	5:03: PM	Chikani, Fatema Z. (Pronoun: She / Her)
Wed, April 29	5	5:05: PM	Munnangi, Pratibha (Pref: Pratibha, Pronoun: she/her)
Wed, May 6	6	4:55: PM	Ngo, Huy (Pref: Michael)
Wed, May 6	6	4:57: PM	Nguyen, Nhat M. (Pronoun: he/him)
Wed, May 6	6	4:59: PM	Naidu, Mahesh (Pref: Mahesh, Pronoun: He/Him)
Wed, May 6	6	5:01: PM	Rabiah, Yaqoub Y. (Pref: Yaqoub)
Wed, May 6	6	5:03: PM	Sabti, Saleh M. (Pref: Saleh, Pronoun: He/Him)
Wed, May 6	6	5:05: PM	Alali, Manaf A.
Wed, May 13	7	4:55: PM	Stanton, Andrew J. (Pref: Andrew, Pronoun: he/him)
Wed, May 13	7	4:57: PM	Bandla, Karthik
Wed, May 13	7	4:59: PM	Alrandi, Dawood A.
Wed, May 13	7	5:01: PM	Premarajan, Roshan B. (Pref: Roshan, Pronoun: hr/him)
Wed, May 13	7	5:03: PM	Vedantam, Venkata Krishna Kumar (Pronoun: His/Him)
Wed, May 13	7	5:05: PM	Esmail, Saleh A. (Pref: Sal, Pronoun: He/Him)

Wed, May 20	8	4:55: PM	Mills, Camden J. (Pref: Camden, Pronoun: He/Him)
Wed, May 20	8	4:57: PM	Day, Elliott F. (Pronoun: he/him)
Wed, May 20	8	4:59: PM	Mahajan, Ria (Pref: Ria)
Wed, May 20	8	5:01: PM	Khaddoori, Ibrahim A. (Pref: Ibrahim)
Wed, May 20	8	5:03: PM	AlQuzweni, Hussain A. (Pref: Hussain, Pronoun: he/him)
Wed, May 20	8	5:05: PM	Alotaibi, Mohammed M. (Pref: Mohammed)
Wed, May 27	9	4:55: PM	Wyse, Ian A. (Pronoun: he/him/his)
Wed, May 27	9	4:57: PM	Vanukuri, Sri Chandana Reddy
Wed, May 27	9	4:59: PM	Thakur, Amogh Vinod
Wed, May 27	9	5:01: PM	Kyatham, Vanisri (Pronoun: she/her)
Wed, May 27	9	5:03: PM	Vanamala, Naveen Babu (Pronoun: HE/HIM)
Wed, May 27	9	5:05: PM	Mallick, Devdutt S. (Pref: Devdutt, Pronoun: He/Him)
Wed, May 27	9	5:07: PM	Alshaiji, Mohammad A. (Pronoun: He/Him)
Wed, June 3	10	4:55: PM	Ramirez, Jose A. (Pronoun: he/him)
Wed, June 3	10	4:57: PM	Venkatesha Murthy, Veekshith (Pref: Veekshith, Pronoun: he, him)
Wed, June 3	10	4:59: PM	Handwerger, Adam L.
Wed, June 3	10	5:01: PM	Akula, Ranjith (Pronoun: he/him)
Wed, June 3	10	5:03: PM	Mickelson, Christian I. (Pref: Christian, Pronoun: He/Him)
Wed, June 3	10	5:05: PM	Shamszadeh, Navid (Pref: Navid, Pronoun: he/him)
Wed, June 3	10	5:07: PM	De Los Santos Razo, Jorge J. (Pronoun: He/Hims)